

# 20250703 日报

## 今日工作内容

### 1、actor 模型与 virtual actor 模型对比调研

#### a. 概念

##### i. actor 模型

1. 每个 actor 是一个独立计算实体，包含状态 (state)、行为 (behavior)、邮箱 (mailbox)
2. actor 之间通过异步消息传递交互，消息不可变且 FIFO 顺序处理，邮箱作为缓冲队列
3. 单线程处理消息，即串行处理，天然避免锁竞争和共享内存问题（比如死锁等问题）
4. 需显式管理 actor 的创建、销毁和异常恢复

##### ii. virtual actor 模型

1. virtual actor 又称 grain，它的物理激活/回收由运行时自动管理，即透明激活
2. 调用者无需感知 grain 物理位置，直接用逻辑 id 就行
3. 运行时自动重建崩溃的 actor，业务代码无需处理
4. 闲置 actor 自动释放内存，请求到达时动态激活（不管请求的 grain 是否存在）

#### b. 框架

##### i. actor 模型：Akka/Erlangee

##### ii. virtual actor 模型：Orleans/Dapr/ProtoActor

#### c. c)适用场景

- i. actor 模型：高并发事务（需协调多个 actor 避免中间状态）、流处理系统、物联网平台等
- ii. virtual actor 模型：游戏后端服务（如微软游戏《Halo 4/5》的云服务）、无状态 API 聚合、快速开发分布式系统

### 2、基于 actor 模型原理和工蜂上类似的例程自己写了一个 RedisActor

```

package actor

import (
    "context"
    "fmt"
    "sync"
    "sync/atomic"
    "time"

    "git.code.oa.com/trpc-go/trpc-database/redis"
)

const (
    ActorCreating      int32 = iota // actor 创建中
    ActorInitializing           // actor 初始化中
    ActorRunning               // actor 运行中
    ActorStopping              // actor 停止中
    ActorStopped               // actor 已停止
)

type CommandMsg struct {
    Cmd      string
    Key      string
    Content  interface{}
    resultCh chan<- Result // 单向通道, 只能发送结果
}

type Result struct {
    Value interface{} // Redis 操作返回值
    Err   error        // 错误信息
}

type (
    RedisActor interface {
        Init(ctx context.Context, proxy redis.Client) error
        Post(msg CommandMsg) error
        Stop()
        Id() uint64
        Status() int32
        GetConfig() *RedisActorConfigs
        Call(ctx context.Context, msg CommandMsg) (Result, error)
    }
)

```

- 基于自己构建的 RedisActor 模型进行 redis 的 get 请求测试：

```

// 普通客户端，无并发测试
func Redis_Client() {
    // 初始化 Redis 客户端
    proxy := redis.NewClientProxy("trpc.redis.hello.service")

    // 创建并初始化 Actor
    redisactor := actor.NewRedisActor(1)
    if err := redisactor.Init(context.Background(), proxy); err !=
nil {
        log.Fatal("初始化失败: ", err)
    }
    defer redisactor.Stop()

    // 构造消息并同步调用
    msg := actor.CommandMsg{
        Cmd: "GET",
        Key: "name",
    }

    // 带超时控制的同步调用
    ctx, cancel := context.WithTimeout(context.Background(),
2*time.Second)
    defer cancel()

    res, err := redisactor.Call(ctx, msg)
    if err != nil {
        log.Fatal("调用失败: ", err)
    }
    // 处理 Redis 返回结果
    switch {
    case res.Err == redis.ErrNil:
        fmt.Println("键不存在")
    case res.Err != nil:
        log.Fatal("Redis 错误: ", res.Err)
    default:
        val, _ := res.Value.([]byte) // 类型断言
        fmt.Printf("获取成功: %s\n", val)
    }
}

```

```
● [root@drewjjli-1t70osepea test]# ./test_redisactor  
获取成功: drewjjli
```

- 基于自己构建的 RedisActor 模型进行 redis 的并发 GET 请求测试，10000 个并发协程同时请求 get 指令：
  - redisactor 模型

```

func Redis_Client() {
    // 初始化 Redis 客户端
    proxy := redis.NewClientProxy("trpc.redis.hello.service")
    redisactor := actor.NewRedisActor(1)
    if err := redisactor.Init(context.Background(), proxy); err !=
nil {
        log.Fatal("初始化失败: ", err)
    }
    defer redisactor.Stop()

    // 并发控制配置
    const concurrency = 10000 // 1000 个并发协程
    var wg sync.WaitGroup
    wg.Add(concurrency)
    start := time.Now() // 记录测试开始时间
    // 创建带缓冲的结果通道（避免阻塞）
    results := make(chan string, concurrency)
    // 启动 1000 个协程
    for i := 0; i < concurrency; i++ {
        go func(id int) {
            defer wg.Done()
            // 构造相同的 GET 消息
            msg := actor.CommandMsg{
                Cmd: "GET",
                Key: "name",
            }
            // 带超时控制的同步调用
            ctx := context.Background()
            // 发送请求并获取结果
            res, _ := redisactor.Call(ctx, msg)
            results <- fmt.Sprintf("协程%d: 值=%s", id, res.Value)
        }(i) // 传入协程 ID
    }
    wg.Wait()
    close(results)
    // 统计结果
    success, failure := 0, 0
    for res := range results {
        if strings.Contains(res, "错误") || strings.Contains(res, "
不存在") {
            failure++

```

```
[root@drewjjli-1t70osepea test]# ./test_redisactor
2025-07-03 20:50:15.711 INFO test/test_redisactor.go:110 成功率: 100.0%, 成功: 10000, 失败: 0
2025-07-03 20:50:15.711 INFO test/test_redisactor.go:113 测试完成! 总耗时: 1.071969484s, QPS: 9329
```

## ○ mutex 加锁

```
[root@drewjjli-1t70osepea test]# ./test_redismutex
2025-07-03 20:52:37.769 INFO test/test_redismutex.go:83 成功率: 100.0%, 成功: 10000, 失败: 0
2025-07-03 20:52:37.770 INFO test/test_redismutex.go:86 测试完成! 总耗时: 4.690429066s, QPS: 2132
```

从结果来看，actor 模型在某些高并发情况下的耗时有明显改善。我这里对 redis 的映射关系是一个 redis 服务对应一个 actor 模型，这样就自动实现了所有并发请求的串行操作，并且无需加锁。目前我没有对资源消耗进行对比，尚不清楚利用 actor 模型所耗资源和加锁所耗资源的对比数据。如果使用更细的映射关系，actor 模型在延迟和资源方面能有更好的效果，比如在某个键值对较少的 redis 服务中，可以使用一个 key 对应一个 actor，此时就能高并发的对同一个 redis 下的所有 key 进行并发操作，并且不加锁。下一步可以对此 RedisActor 进行路由管理（仅使用逻辑名就能和 actor id 对应，能利用逻辑名自动找到所要的 actor（如果 actor 不存在就会自动创建））、分布式集群（分布式 actor 之间可以互相交互）等功能开发。

## 明日待办：

1. 找到 trpc-go 启动的时候，七彩石插件超时的问题，并找到解决办法。
2. 对齐 virtual actor 的任务需求。